

깃허브 정리 및 몬스터 사 냥 게임

<https://github.com/yunsungchoi56/python>

2026764049 최윤성

main 1 Branch 0 Tags

Go to file Add file Code

yunsungchoi56	Create readme.md	c4b18fe · 3 weeks ago	35 Commits
0306	Update and rename hello.py to 0306/hello.py	3 months ago	
0313	Add files via upload	3 months ago	
0320	Update fact.py	3 months ago	
0403	Create readme.md	2 months ago	
0410	Add files via upload	2 months ago	
0417	Add files via upload	2 months ago	
0508	Add files via upload	last month	
0515	Add files via upload	last month	
0522	Create readme.md	3 weeks ago	
0306 readme.md	Create 0306 readme.md	3 months ago	
0313 readme.md	Create 0313 readme.md	3 months ago	
README.md	Initial commit	3 months ago	

README

About

2026 Python 입문

Readme

Activity

0 stars

0 watching

0 forks

Releases

No releases published

[Create a new release](#)

Packages

No packages published

[Publish your first package](#)

Contributors 1

yunsungchoi56

Languages

Python 100.0%

1주차:파이썬 기초

1장 파이썬 소개 요약

컴퓨터는 하드웨어와 소프트웨어로 구성되며, 다양한 일을 할 수 있는 유연한 기계이다. 컴퓨터에게 일을 시키려면 사람이 자세한 명령어 목록을 작성해야 하는데 이것을 프로그램이라고 한다. 인공지능, 주식 자동매매, 로봇 프로세스 자동화처럼 여러 분야에서 프로그램이 활용된다. 컴퓨터는 반복적이고 지루한 작업을 빠르고 정확하게 처리하는 데 강점이 있다. 컴퓨터는 사람의 언어를 직접 이해하지 못하므로 프로그래밍 언어가 필요하다. 사람이 파이썬 같은 언어로 프로그램을 작성하면 컴파일러나 인터프리터가 이를 컴퓨터가 이해할 수 있는 기계어로 바꿔준다. 문법이 간결하고 배우기 쉬우며 결과를 바로 확인할 수 있는 인터프리터 언어라 초보자에게 적합하다. 또한 라이브러리가 풍부하고 설치가 쉬워 데이터 분석, 인공지능, 웹 개발 등 다양한 분야에 활용된다. 주피터 노트북과 Colab은 코드와 설명을 함께 작성하며 실행할 수 있는 환경이다. 기초 문법으로는 `print()` 함수, 문자열, 계산식 출력이 다뤄진다. 문자열은 큰따옴표나 작은따옴표 안에 작성해야 하며 따옴표가 없으면 오류가 발생한다. `print()` 함수는 문자열이나 계산 결과를 화면에 출력할 때 사용한다.

예를 들어:

```
print("파이썬에 오신 것을 환영합니다.")
```

```
print("2+3=", 2+3)
```

오류 처리에서는 따옴표 누락, 대소문자 오류, 괄호 오류 등이 소개된다. 파이썬은 대소문자를 구분하므로 `print`와 `Print`는 다르게 인식된다. 소스 파일의 확장자는 `.py`이다.

Commits

History for `Python` / `0306` on `main`

Commits on Mar 13, 2026

Update and rename hello.py to 0306/hello.py

yunsungchoi56 authored on Mar 13

Create readme.md

yunsungchoi56 authored on Mar 13

End of commit history for this file

yunsungchoi56 Update and rename hello.py to 0306/hello.py

Code

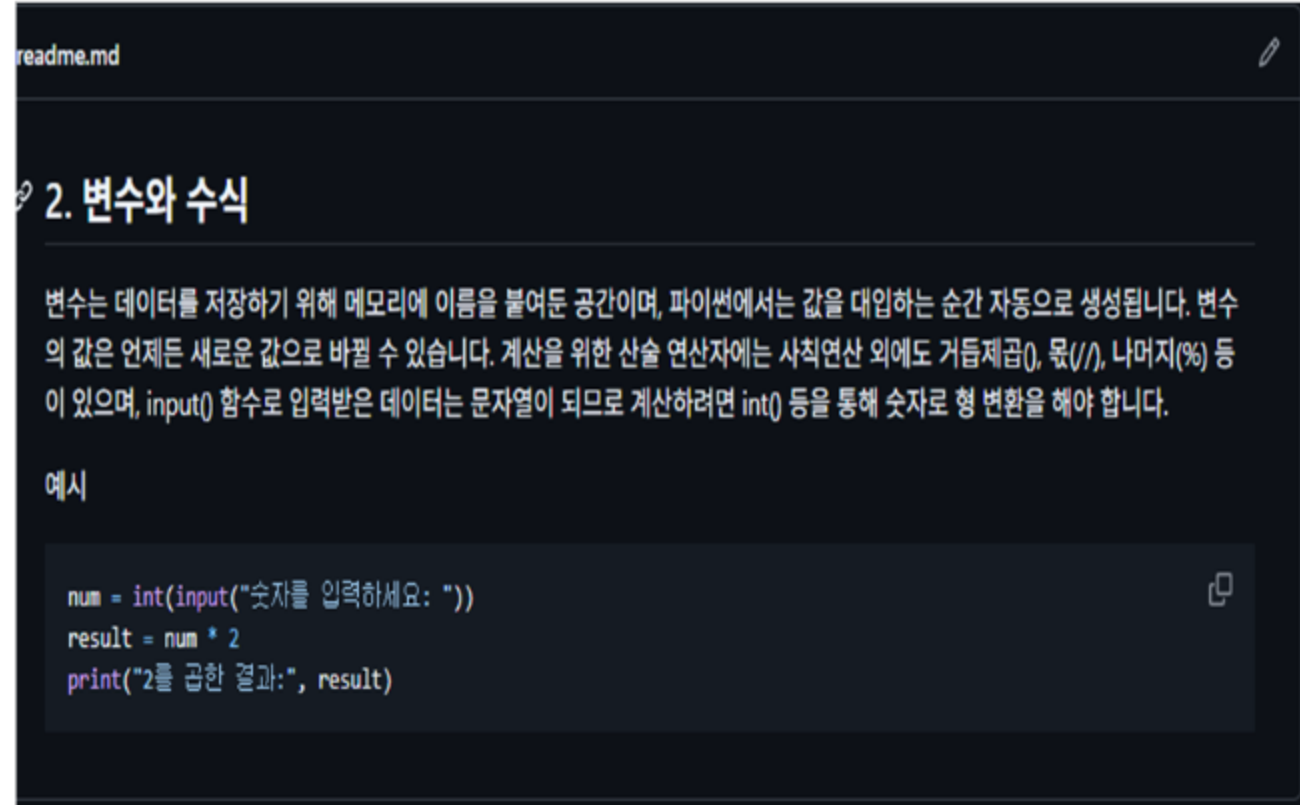
Blame

2 lines (2 loc) · 64 Bytes



```
1 print("Hello python world!")
2 print("Welcome to python world")
```

2주차:변수와 수식



중간점검

1. 변수를 다시 설명해보자.
변수는 데이터를 저장하는 공간이고, 나중에 사용하기 쉽게 이름을 붙여서 관리하는 것이다.
2. 밑변이 10이고 높이가 10인 삼각형의 면적을 계산하는 프로그램을 작성해보자.
base = 10
height = 10
area = base * height / 2

print(f'밑변 {base}, 높이 {height}인 삼각형 면적은 {area}이다.')
3. 변수 이름을 만들 때 지켜야 하는 규칙은 무엇인가?
변수 이름은 알아보기 쉽게 만드는게 좋고, 대문자와 소문자는 다르게 인식된다. 영어랑 숫자, _(언더바)를 사용할 수 있고 중간에 띄어쓰기는 안된다.
4. 변수 이름의 첫 번째 글자로 허용되는 것은 무엇인가?
영어 문자나 _(언더바)만 가능하다.
5. 파이썬에서 고유한 의미를 가지고 있는 단어들을 무엇이라고 하는가?
예약어라고 한다. 이미 기능이 정해진 단어들이고 while, for, def 같은 것들이 있다.
6. 파이썬에는 어떤 자료형이 있는가? 3가지만 말해보자.
int, float, str 이 있다.
7. 파이썬 변수에 정수를 저장하였다가 실수를 저장하는 것도 가능한가? 그 이유는 무엇인가?
가능하다. 파이썬은 동적타입 언어라서 값이 바뀌면 자료형도 같이 바뀐다.
8. 변수의 자료형을 알려면 어떤 함수를 사용하는가?
type()을 사용하면 된다.

0313

Inputfunc.py

circle area.py

readme.md

triangle area.py

과제 (1).hwp

Lab: 별까지의 거리 계산하기

- 지구에서 가장 가까운 별은 어디일까? 정답은 태양이다. 태양 다음으로 가까운 별은 프록시마 센토리(Proxima Centauri) 별이라고 한다. 프록시마 센토리는 지구로부터 km 떨어져 있다고 한다. 빛의 속도로 프록시마 센토리까지 갔다면 시간이 얼마나 걸리는지 직접 계산해보기로 하자. 단위는 광년(light year)이다. 빛의 속도는 이다. 변수를 최대한 많이 사용하여 보자.

```
speed = 300000.0 # 빛의 속도
distance = 40000000000000.0 # 거리
secs = distance / speed # 걸리는 시간, 단위는 초
light_year = secs / (60.0*60.0*24.0*365.0) # 광년으로 변환
print(light_year, "광년")
```

Lab: 원기둥의 부피 계산

```
# 원주율을 나타내는 상수를 정의한다.
PI = 3.14
# 변수를 정의한다.
radius = 5
height = 10
# 부피를 계산한다.
volume = PI * radius * radius * height
# 결과를 출력한다.
print("반지름=", radius, "높이=", height, "원기둥의 부피=", volume)
```

3주차:조건문

```
readme.md

3. 조건문

조건문은 주어진 조건에 따라 프로그램의 실행 흐름을 바꾸는 제어문입니다. if, elif, else 구조를 사용하며, 조건식에는 대소 비교를 하는 관계 연산자와 여러 조건을 묶어주는 논리 연산자(and, or, not)가 사용됩니다. 파이썬에서는 조건문 뒤에 실행할 명령문들을 반드시 동일한 깊이로 들여쓰기해야 같은 블록으로 인식됩니다.

예시

score = 85
if score >= 60:
    print("합격입니다.")
else:
    print("불합격입니다.")
```

Code Blame 10 lines (8 loc) · 160 Bytes

```
1 sum = 0
2
3 for i in range(1,11):
4     sum += i # sum = sum + i
5     if i < 10:
6         print(i, end=" +")
7     else:
8         print(i, end=" = ")
9
10 print(sum)
```

0320

- Forex01.py
- fact.py
- list.py
- matchcase.py
- range.py
- read.me
- sansoo.py
- shipping_cost.py
- sqrt.py
- yoonyun.py

```
1 mylist = [1,2,3,4,5]
2
3 mylist[0] = 0
4 mylist[-1] = 9
5
6 print(mylist)
```


4주차:반복문

readme.md

4. 반복문

반복문은 동일하거나 유사한 작업을 여러 번 수행할 때 사용하며, 컴퓨터의 빠른 처리 능력을 극대화하는 문법입니다. 횟수가 정해진 반복에는 숫자 범위를 만들어주는 range() 함수와 for문을 주로 사용하고, 특정 조건이 참인 동안 계속 반복할 때는 while문을 사용합니다. while문을 쓸 때는 조건이 영원히 참이 되어 멈추지 않는 '무한 루프'에 빠지지 않도록 종료 조건을 잘 제어해야 합니다.

예시

```
for i in range(3):  
    print("방문을 환영합니다!")
```

Lab: 대화하는 프로그램 만들기

```
C:\Users\USER> CodeDrive > 문서 > Untitled 1.py > ...  
1 print("안녕하세요.")  
2 name = input("이름이 어떻게 되시나요? ")  
3 print("안녕하세요, " + name + "님!")  
4 print("이름이 길이는 다음과 같습니다: " + len(name))  
5 age = int(input("나이가 어떻게 되시나요? "))  
6 next_year_age = age + 3  
7 print("다음 해엔 " + str(next_year_age) + "이 되실거예요.")  
8  
9  
10  
11  
12  
13  
14  
15  
16 C:\Users\USER>
```

Lab: BMI 계산하기

```
C:\Users\USER> CodeDrive > 문서 > Untitled 1.py > ...  
1 weight = float(input("몸무게를 kg 단위로 입력하세요: "))  
2 height = float(input("키를 m 단위로 입력하세요: "))  
3 bmi = weight / (height**2)  
4 print("당신의 BMI는: " + str(bmi))  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16 C:\Users\USER>
```

Lab: 별까지의 거리 계산하기

```
C:\Users\USER> CodeDrive > 문서 > Untitled 1.py > ...  
1 speed = 1000000000  
2 distance = 4000000000000000  
3 secs = distance / speed  
4 light_year = secs / (60*60*24*365*1000)  
5 print(light_year, "광년")  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16 C:\Users\USER>
```

Lab: 원기둥의 부피 계산

```
C:\Users\USER> CodeDrive > 문서 > Untitled 1.py > ...  
1 radius = 5  
2 height = 10  
3 volume = 3.14 * radius * radius * height  
4 print("원기둥의 부피는: " + str(volume) + "입니다.")  
5  
6  
7  
8  
9  
10  
11  
12  
13  
14  
15  
16 C:\Users\USER>
```

5주차:함수

5장 함수 요약

함수는 특정 작업을 수행하는 명령어들의 모음에 이름을 붙인 것이다. 반복해서 사용하는 코드를 함수로 만들면 프로그램을 더 깔끔하고 효율적으로 작성할 수 있다.

함수는 `def`로 정의한다.

```
def hello():  
    print("안녕하세요")
```

함수를 실행하려면 함수 이름을 호출한다.

```
hello()
```

함수에는 값을 전달할 수 있는데, 이를 매개변수라고 한다.

```
def greet(name):  
    print(name, "님 안녕하세요")
```

함수는 결과값을 돌려줄 수도 있다. 이때 `return`을 사용한다.

```
def add(x, y):  
    return x + y
```

함수는 코드 재사용, 프로그램 구조화, 오류 감소에 도움을 준다. 이 장에서는 사각형 그리기, 막대그래프 그리기, 계산 함수 만들기 등을 통해 함수의 필요성과 사용법을 익혔다.

Commits

History for `Python / 0410` on [main](#)

All users

All time

Commits on Jun 10, 2026

Update readme.md

ygdn12 authored 2 hours ago

Verified

a0438b6



Commits on Apr 15, 2026

Add files via upload

ygdn12 authored on Apr 15

Verified

71e03ea



Create readme.md

ygdn12 authored on Apr 15

Verified

fa7b8d8



6주차:파이썬 리스트,자료구조

6. 파이썬 자료구조 I: 리스트

리스트는 여러 개의 데이터를 대괄호[] 안에 순서대로 나열하여 저장하는 자료구조입니다. 각 데이터의 위치를 나타내는 번호인 '인덱스'는 0부터 시작하며, 이를 통해 특정 위치의 값을 읽거나 수정할 수 있습니다. append()로 새 값을 추가하거나 sort()로 정렬할 수 있으며, 리스트 안에 또 다른 리스트를 넣어 행렬 형태의 2차원 리스트를 만들 수도 있습니다.

예시

```
fruits = ["사과", "바나나"]  
fruits.append("포도")  
print(fruits[0])
```

0417

readme.md

과제 (1).hwp

Commits

History for [Python](#) / [0417](#) / [과제 \(1\).hwp](#) on [main](#)

Commits on Apr 25, 2026

Add files via upload

yunsungchoi56 authored on Apr 25

End of commit history for this file

7주차:파이썬자료구조 활용,튜플,딕셔너리, 세트, 문자열

7장 자료구조 || 요약

이 장에서는 리스트 외의 자료구조인 튜플, 세트, 딕셔너리, 문자열 연산을 배운다. 튜플은 리스트와 비슷하지만 한 번 만들면 값을 변경할 수 없다.

```
fruits = ("apple", "banana", "grape")
```

튜플은 변경되면 안 되는 데이터를 저장할 때 사용한다. 항목이 하나인 튜플을 만들 때는 쉼표가 필요하다.

```
single = ("apple",)
```

세트는 중복을 허용하지 않고 순서가 없는 자료구조이다.

```
s = {1, 2, 3}
```

세트는 중복 제거, 집합 연산에 유용하다. 합집합, 교집합, 차집합 등을 구할 수 있다. 딕셔너리는 키와 값의 쌍으로 데이터를 저장한다.

```
phone = {"KIM": "123-4567"}
```

키를 이용해 값을 찾을 수 있으므로 연락처, 사전, 회원 정보처럼 이름과 값이 연결된 자료를 저장할 때 좋다.

```
phone["KIM"]
```

문자열도 시퀀스 자료형이므로 인덱싱, 슬라이싱, 반복, 검색이 가능하다. 문자열 메서드로는 upper(), lower(), split(), replace(), find() 등이 있다. 이 장에서는 연락처 추가, 삭제, 검색, 출력 기능을 가진 간단한 연락처 관리 프로그램을 만들었다.

Commits

History for [Python](#) / [0508](#) / [0508.hwp](#) on [main](#)

🔗 Commits on May 15, 2026

Add files via upload

👤 yunsungchoi56 authored last month

🔗 End of commit history for this file

📁 0508

📄 0508.hwp

📄 readme.md

8주차: 객체 지향 패러다임 도입 (클래스)

8장 객체와 클래스 요약

객체지향 프로그래밍은 데이터와 함수를 하나로 묶어 객체로 만들고, 객체들이 서로 상호작용하며 프로그램을 구성하는 방식이다. 객체는 속성과 동작을 가진다. 예를 들어 자동차 객체는 색상, 모델, 속도 같은 속성을 가지고, 달리기, 멈추기, 방향 바꾸기 같은 동작을 가진다. 클래스는 객체를 만들기 위한 설계도이다. 클래스로부터 만들어진 실제 객체를 인스턴스라고 한다.

```
class Car:
    def __init__(self, color, model):
        self.color = color
        self.model = model
        self.speed = 0
```

`init()`은 객체가 생성될 때 자동으로 실행되는 생성자이다. `self`는 객체 자기 자신을 의미한다. 객체 안에 들어 있는 함수는 메서드라고 한다.

```
def speed_up(self):
    self.speed += 10
```

파이썬에서는 정수, 문자열, 리스트 등 거의 모든 것이 객체이다. 예를 들어 문자열에서 `.upper()`를 호출할 수 있는 것도 문자열이 객체이기 때문이다. 객체지향의 핵심 개념으로는 캡슐화, 클래스, 객체, 속성, 메서드 등이 있다. 이 장에서는 자동차 클래스를 만들어 속도, 색상, 모델을 저장하고 속도를 변경하는 프로그램을 작성했다.

0515

readme.md

과제0515.hwp

Commits

History for [Python](#) / [0515](#) / [readme.md](#) on [main](#)

Commits on May 15, 2026

Create [readme.md](#)

 [yunsungchoi56](#) authored last month

End of commit history for this file

몬스터 사냥 게임


```

35 # 3. 도망 선택
36 elif choice == "3":
37     # 50% 확률로 도망 성공
38     if random.random() < 0.5:
39         print("\n🏃 성공적으로 도망쳤습니다! 게임을 종료합니다.")
40         return
41     else:
42         print("\n❌ 도망치는데 실패했습니다! 몬스터가 붙잡았습니다.")
43
44 else:
45     print("\n⚠️ 잘못된 입력입니다. 당황해서 어버버하는 사이에...")
46
47 # 몬스터가 아직 살아있다면 플레이어를 공격
48 if monster_hp > 0:
49     time.sleep(0.8)
50     monster_damage = random.randint(monster_atk - 2, monster_atk + 4)
51     player_hp -= monster_damage
52     print(
53         f"\n🐉 {monster_name}의 반격! {monster_damage}의 피해를 입었습니다."
54     )
55     time.sleep(0.8)
56
57 # 게임 결과 판정
58 print("\n=====")
59 if player_hp > 0:
60     print(f"\n🎉 축하합니다! [{monster_name}](을)를 물리쳤습니다! 🎉")
61 else:
62     print("\n💀 눈앞이 캄캄해졌다... 게임 오버되었습니다.")
63 print("=====")
64
65 # 게임 실행
66 if __name__ == "__main__":
67     monster_hunting_game()

```

=====

❌몬스터 사냥꾼 게임에 오신 것을 환영합니다! ❌

=====

🔊야생의 [심연의 드래곤](이)가 나타났습니다!

❤️내 체력: 100/100 | 🍷포션: 3개
 🐉심연의 드래곤 체력: 150

1. 공격하기 ❌
 2. 포션 마시기 🍷(체력 +40)
 3. 도망치기 🏃
 무엇을 하시겠습니까? (1/2/3): 1

❌몬스터를 공격합니다!
 ✨크리티컬 히트!!! 22의 강력한 피해를 입혔습니다!
 🐉심연의 드래곤의 반격! 12의 피해를 입었습니다.

❤️내 체력: 88/100 | 🍷포션: 3개
 🐉심연의 드래곤 체력: 128

1. 공격하기 ❌
 2. 포션 마시기 🍷(체력 +40)
 3. 도망치기 🏃
 무엇을 하시겠습니까? (1/2/3): 1

❌몬스터를 공격합니다!
 ✨13의 피해를 입혔습니다.
 🐉심연의 드래곤의 반격! 13의 피해를 입었습니다.

❤️내 체력: 75/100 | 🍷포션: 3개
 🐉심연의 드래곤 체력: 115

1. 공격하기 ❌
 2. 포션 마시기 🍷(체력 +40)
 3. 도망치기 🏃
 무엇을 하시겠습니까? (1/2/3): 1

느낀점

이번 프로젝트에서 파이썬의 핵심 문법인 변수, 조건문, 반복문, 함수 등을 활용해 프로그램 작성을 진행해 보았습니다. 교재로만 공부할 때와 달리, 직접 기획부터 구현까지 마주해보니 프로그래밍은 직접 부딪혀보며 배우는 것이 가장 효과적이라는 것을 느꼈습니다.

개발 과정에서 코드가 생각대로 동작하지 않아 시행착오를 겪기도 했지만, 함수를 통해 코드를 기능별로 분리하고 차근차근 로직을 점검하면서 문제 해결 능력을 기를 수 있었습니다. 특히 사용자가 편리하게 사용할 수 있도록 버튼 배치나 안내 메시지 같은 작은 디테일까지 고민해 보는 과정이 무척 뜻깊었습니다.

기본 문법들이 합쳐져 하나의 완성도 있는 프로그램으로 작동하는 것을 보며 큰 성취감을 느꼈고, 파이썬이라는 언어의 매력을 알게 되었습니다. 이번 경험을 발판 삼아 다음에는 더 다양한 기능을 추가하거나 새로운 프로그램 개발에도 도전해 보고 싶습니다.

나의 예상 점수

GITHUB점수 15/15
프로젝트 발표 15/15
총합 30/30